Seguros Constitucion

CONTRALORÍA CORPORATIVA

INFORME PREVIO DE AUDITORÍA

Desarrollo

Auditoría	AUD-2026-0003	Expediente	EXP-2026-0004
Unidad auditada	SISTEMAS (unidad)	Modalidad	Presencial
Período revisado	2026-09-28 a 2026-10-06	Fecha del informe	2026-10-07

1. Objetivo y alcance

Levantamiento de informacion

Levantamiento de informacion

Procedimientos aplicados

Levantamiento de informacion

Equipo auditor

Adrian Alejandro Munoz Avila (lider)

2. Resultados por cédula

Cédula	Ítems	Excep.	Nota auto.	Nota final	Calificación
Auditoría de Sistemas — ERP / TI	48		68.4	68.4	Aceptable
CALIFICACIÓN GLOBAL	48		68.4	68.4	Aceptable

Escala: Satisfactorio ≥ 90; Necesita mejoras ≥ 60; Insatisfactorio por debajo.

3. Observaciones

Observación 1: Manual de Gestion de Cambios	**Riesgo ALTO**
Condición: Totalmente desactualizado	
Criterio: No posee informacion ni la unidad sigue este manual	
Causa: Alta Rotacion del personal y falta de politicas	
Efecto: No cumplir con los procedimientos, ni normas dentro del desarrollo para un cambio establecido	
Recomendación: Hacer mesa de trabajo para actualizacion de politicas, incluyendo este manual	

INFORME PREVIO

Observación 2: Metodologia para la gestión integral de proyectos de TI	**Riesgo MEDIO**
Condición: Es inexistente	
Criterio: No posee alguna metodologia formal documentada ni aprobada	
Recomendación: Creacion de una metodologia o documentar gestión integral de proyectos de TI	

Observación 3: Documentacion de políticas que rijan explícitamente el ciclo de vida del desarrollo de software (SDLC)	**Riesgo ALTO**
Condición: Inexistente	
Criterio: No posee ninguna documentacion	
Efecto: Exposicion al software desfasado, y brechas de seguridad	
Recomendación: Mesa de trabajo para documentacion del desarrollo y software actual. Creacion de Politicas	

Observación 4: Analisis de Ciberseguridad para el previo desarrollo	**Riesgo ALTO**
Condición: No posee / Inexistente	
Criterio: No posee / Inexistente	
Causa: No posee / Inexistente	
Efecto: Exposicion al software malicioso, y vulnerabilidades de seguridad	
Recomendación: Analisis profundo, junto con el equipo de Ciberseguridad para el codigo existente, y en desarrollo	

Observación 5: Trazabilidad en la matriz de prueba	**Riesgo ALTO**
Condición: No posee / Inexistente	
Criterio: No posee / Inexistente	
Causa: No posee / Inexistente	
Efecto: 1. Riesgos de Calidad y Operación (El Software Falla) • Brechas de Cobertura (Casos "Ciegos"): Quedarán requisitos sin probar simplemente porque el equipo de control de calidad (QA) no sabía que existían o no entendió cómo debían funcionar. • Fugas de Errores a Producción: Los errores graves llegarán al usuario final, afectando la reputación de la empresa y la continuidad del negocio. • Defectos Recurrentes (Regresión): Al hacer cambios o actualizaciones, no sabrás qué pruebas volver a ejecutar, lo que provocará que funciones que ya servían dejen de funcionar. 2. Riesgos de Tiempos y Costos (Desperdicio de Recursos) • Efecto "Teléfono Descompuesto": Los desarrolladores programarán basándose en suposiciones y QA probará basándose en suposiciones distintas. Esto genera retrabajo constante y discusiones sobre si un comportamiento es un "error" o una "característica". • Retrasos en las Entregas: Al no tener criterios claros, la fase de pruebas se vuelve caótica y se extiende indefinidamente, retrasando la salida al mercado (Time-to-Market). • Desperdicio de Esfuerzo: Se gastará tiempo diseñando pruebas repetitivas o probando funciones irrelevantes, mientras se descuidan los flujos críticos del negocio.	
Recomendación: 1. Establece el filtro de "Definición de Listo" (DoR) 2. Adopta la Matriz de Trazabilidad en una hoja compartida (Excel/Sheets) 3. Automatizar la rastreabilidad (Si ya usan herramientas de gestión, si no implementar una)	

INFORME PREVIO

Observación 6: Diagramas de flujo de datos (Data Flow Mapping) en la arquitectura inicial del proyecto	**Riesgo ALTO**
Condición: No posee / Inexistente	
Criterio: No posee / Inexistente	
Causa: No posee / Inexistente	
Efecto: . Riesgos de Seguridad y Cumplimiento (El más crítico) • Puntos ciegos de ciberseguridad: No se pueden identificar las zonas donde los datos viajan desprotegidos (en texto plano), facilitando la interceptación de información sensible por atacantes. • Incumplimiento legal y multas: Sin un mapa de datos, es imposible cumplir con normativas de privacidad (como GDPR o regulaciones locales). Si no sabes exactamente dónde se almacenan o por dónde pasan los datos de los clientes, te expones a penalizaciones masivas. 2. Riesgos de Arquitectura y Rendimiento (Problemas Técnicos) • Cuellos de botella invisibles: Al programar a ciegas, se suelen crear conexiones redundantes o circulares (por ejemplo, tres servicios consultando la misma base de datos al mismo tiempo), lo que satura los servidores y ralentiza la aplicación. • Incompatibilidad de integración: Al integrar sistemas de terceros o APIs, el equipo descubrirá demasiado tarde que los formatos de datos (JSON, XML, tipos de variables) no coinciden, obligando a rehacer código desde cero. 3. Riesgos de Negocio y Costos (Impacto Financiero) • Retrabajo masivo: Si las reglas sobre cómo se transforma el dato cambian a mitad del proyecto, el costo de modificar una base de datos ya construida es hasta 10 veces mayor que haberlo diseñado bien en el papel.	
Recomendación: etener el desarrollo activo de nuevas funciones durante 3 a 5 días y realizar un taller de mapeo técnico (Data Flow Workshop) utilizando la metodología de niveles progresivos. No intentes mapear todo el ecosistema de golpe; desglósalo de la siguiente manera: [Nivel 0: Contexto General] ■■> [Nivel 1: Procesos Clave] ■■> [Nivel 2: Detalle Técnico]	

INFORME PREVIO

Observación 7: Aprobación formal del diseño por parte del patrocinador antes de iniciar la etapa de codificación	**Riesgo ALTO**
Condición: No posee / Inexistente	
Criterio: No posee / Inexistente	
Causa: No posee / Inexistente	
Efecto: 1. Riesgos Legales y Financieros (El impacto directo) • Multas multimillonarias y sanciones: Regulaciones internacionales como el GDPR (Europa) o normativas locales de protección de datos imponen penalizaciones severas que pueden alcanzar un porcentaje de la facturación anual de la empresa. • Demandas colectivas: Si el sistema sufre una brecha de seguridad y expone datos sensibles de clientes que no fueron protegidos desde el diseño, la empresa enfrentará litigios masivos. • Clausura de operaciones: En sectores altamente regulados (Banca, Finanzas, Salud), los entes reguladores tienen la facultad de suspender las licencias de operación o bloquear la aplicación de forma inmediata si no cumple con las leyes de cumplimiento. 2. Riesgos Técnicos y de Arquitectura (La "Deuda de Cumplimiento") • Rediseño forzado y costoso (Retrofitting): Intentar "parchar" la privacidad en un software que ya está programado es extremadamente complejo. Implementar funciones obligatorias como el Derecho al Olvido (eliminar permanentemente los datos de un usuario de todas las tablas e históricos) o el consentimiento explícito cuando la base de datos ya está construida, puede obligar a reescribir hasta el 40% del código del sistema. • Fugas de datos por diseño defectuoso: Si no se define desde el diseño qué datos deben ir encriptados en tránsito y en reposo, o quién tiene permisos para verlos, los desarrolladores guardarán datos sensibles (como contraseñas, tarjetas de crédito o identificaciones) en texto plano o en archivos de registro (logs) expuestos. 3. Riesgos de Reputación y Negocio • Pérdida total de confianza: Una sola filtración de datos destruye la reputación de una marca en horas. Los clientes migrarán de inmediato a competidores que sí garanticen que su información está segura.	
Recomendación: Adoptar el marco de Privacidad desde el Diseño (Privacy by Design) y el principio de Seguridad por Defecto (Security by Default), transformando el cumplimiento en un habilitador técnico y no en un trámite burocrático.	

INFORME PREVIO

Observación 8: Documentacion y gestion de las dependencias de integración entre microservicios interconectados	**Riesgo ALTO**
Condición: No posee / Inexistente	
Criterio: No posee / Inexistente	
Causa: No posee / Inexistente	
Efecto: 1. Riesgos de Operación y Estabilidad (El "Efecto Dominó") • Fallas en cascada: Si el Microservicio A depende del Microservicio B de forma síncrona y sin control, una caída u obsolescencia en B tumbará de inmediato a A y a todos los servicios que estén detrás. Un pequeño error en un servicio secundario puede paralizar la plataforma entera. • Pesadilla de resolución de problemas (Debugging): Cuando ocurra un error en producción, el equipo de soporte técnico perderá horas o días tratando de descifrar qué servicio originó la falla, pasándose la responsabilidad de un equipo a otro ("el problema es de tu API", "no, es de la tuya"). 2. Riesgos de Desarrollo y Despliegue (Bloqueos constantes) • Pérdida de despliegue independiente: El objetivo de los microservicios es que cada equipo pueda subir cambios a producción de forma autónoma. Sin gestión de dependencias, descubrirás que para desplegar el servicio A, necesitas coordinar y desplegar al mismo tiempo los servicios B, C y D. • Ruptura de contratos (Breaking Changes): Un equipo modificará la estructura de los datos que devuelve su API (por ejemplo, cambiar el nombre de un campo de user_id a id) sin saber que otros tres microservicios dependían de ese campo exacto. El resultado: código roto en producción al instante. 3. Riesgos de Rendimiento (Latencia Oculta) • Llamadas circulares e ineficientes: Al no mapear las dependencias, los desarrolladores terminan creando flujos donde el Servicio A llama al B, el B llama al C, y el C vuelve a llamar al A para validar un dato. Esto degrada drásticamente la velocidad de la aplicación y dispara el consumo de recursos en la nube.	
Recomendación: La recomendación fundamental es romper el acoplamiento fuerte mediante el uso de Contratos Automáticos y Observabilidad, asegurando que el mapa de dependencias se genere de forma viva y no en un documento estático que se queda obsoleto al día siguiente.	

Observación 9: Análisis Estático de Seguridad (SAST) de forma automatizada en el pipeline de construcción	**Riesgo ALTO**
Condición: No posee / Inexistente	
Criterio: No posee / Inexistente	
Causa: No posee / Inexistente	
Efecto: 1. Riesgos de Seguridad y Operación (Vulnerabilidades en Producción) 2. Riesgos de Costos y Eficiencia (La regla del costo del defecto) 3. Riesgos de Gobernanza y Cultura de TI	
Recomendación: La recomendación inmediata es implementar la filosofía de "Shift Left Security" (Mover la seguridad a la izquierda). Esto significa integrar el análisis de seguridad en la etapa más temprana posible del ciclo de desarrollo: justo cuando el código se compila	

INFORME PREVIO

Observación 10: Análisis de Composición de Software (SCA) para identificar vulnerabilidades conocidas en librerías de terceros	**Riesgo ALTO**
Condición: No posee / Inexistente	
Criterio: No posee / Inexistente	
Causa: No posee / Inexistente	
Efecto: 1. Riesgos de Seguridad (Exposición Total) • Ataques automatizados a vulnerabilidades públicas (CVEs): Los ciberdelincuentes no buscan hackear tu código desde cero; escanean aplicaciones buscando librerías antiguas con fallas públicas registradas en la base de datos NVD (National Vulnerability Database). Si usas una versión vulnerable, tu sistema es una puerta abierta. • Ataques a la Cadena de Suministro (Supply Chain Attacks): Inyección de código malicioso en paquetes legítimos de plataformas como npm, NuGet, PyPI o Maven. Sin un SCA, tu pipeline descargará e integrará estos paquetes infectados directamente a tus servidores de producción. 2. Riesgos Legales y de Cumplimiento (Propiedad Intelectual) • Infracción de Licencias (License Compliance): El uso de librerías con licencias de tipo "copyleft" restrictivas (como GPL) puede obligar legalmente a tu empresa a liberar el código fuente de toda tu propiedad intelectual al dominio público. SCA bloquea componentes cuyas licencias violen las políticas del negocio. 3. Riesgos Operativos (Obsolescencia Tecnológica) • Parálisis por Deuda Técnica: El software de terceros envejece rápido. Si dejas pasar años sin actualizar librerías, cuando finalmente intentes hacerlo para parchar una falla, los cambios de versión serán tan drásticos (breaking changes) que romperán la aplicación completa, exigiendo semanas de retrabajo.	
Recomendación: La recomendación fundamental es automatizar el descubrimiento y parcheo de dependencias mediante herramientas SCA integradas directamente en el flujo de trabajo de desarrollo, pasando de un enfoque reactivo a uno preventivo.	

Observación 11: Línea de flujo bloquea automáticamente la compilación si las herramientas de escaneo detectan vulnerabilidades de riesgo crítico	**Riesgo ALTO**
Condición: No posee / Inexistente	
Criterio: No posee / Inexistente	
Causa: No posee / Inexistente	
Efecto: 1. Riesgos de Automatización y Error Humano (Despliegues por Accidente) • Aprobaciones por inercia o urgencia: Si el pipeline solo emite una advertencia o un reporte en PDF pero permite que la compilación termine con éxito, un desarrollador presionado por una fecha de entrega ignorará la alerta con la promesa de "arreglarlo después". Ese "después" rara vez llega. • Automatización ciega: Muchas empresas configuran sus sistemas para que, si la compilación es exitosa, el artefacto se suba de inmediato a un servidor de pruebas o preproducción. Si no se bloquea la construcción, estarás distribuyendo software inseguro por tus entornos internos automáticamente. 2. Riesgos de Visibilidad y Auditoría • Falsos positivos acumulados: Cuando los pipelines no bloquean, los desarrolladores se acostumbran a ver las alertas de seguridad en "rojo" o "amarillo" constantemente. Esto genera fatiga de alertas, provocando que cuando ocurra una vulnerabilidad verdaderamente destructiva, pase desapercibida entre el ruido. • Falta de responsabilidad (Accountability): Sin un bloqueo del sistema, no queda un registro estricto de por qué se permitió avanzar un código defectuoso, lo que rompe la trazabilidad ante auditorías de seguridad (como ISO 27001 o SOC 2).	
Recomendación: Configurar el pipeline de CI/CD para que actúe como un "Guardián Automatizado" mediante políticas de fallo estricto (Fail-Fast Policy), asegurando que la seguridad sea un requisito obligatorio para compilar y no una sugerencia opcional.	

INFORME PREVIO

Observación 12: Bóvedas centralizadas para impedir la incrustación de credenciales estáticas o secretos en el código fuente	**Riesgo ALTO**
Condición: No posee / Inexistente	
Criterio: No posee / Inexistente	
Causa: No posee / Inexistente	
Efecto: 1. Riesgos de Seguridad y Exposición (El "Efecto Historial") • Credenciales inborrables en el historial de Git: Aunque un desarrollador se dé cuenta del error y borre la contraseña del código en un nuevo cambio (commit), la credencial permanece visible para siempre en el historial de versiones de Git. Borrarla de raíz requiere reconstruir el historial del repositorio, lo cual es complejo y detiene el trabajo del equipo. • Fugas masivas en repositorios públicos: Si por error un repositorio privado se configura como público, o si el código se comparte con un proveedor externo, los secretos quedan expuestos a bots automatizados que escanean internet buscando llaves de AWS o bases de datos para secuestrar información en cuestión de minutos. 2. Riesgos de Operación y Mantenimiento (Pesadilla de Rotación) • Imposibilidad de rotar contraseñas: Si una base de datos es comprometida y debes cambiar la contraseña de urgencia, tendrás que buscar en decenas de microservicios dónde estaba escrita esa credencial, modificar el código de cada uno, volver a compilar y redesplegar todo. Esto transforma un incidente de 5 minutos en una crisis de días. • Falta de control de acceso (Principle of Least Privilege): Cualquier desarrollador, QA o practicante que tenga acceso al código fuente tendrá acceso automático a las contraseñas de producción, violando las normas básicas de segregación de funciones. 3. Riesgos de Cumplimiento Normativo • Reprobación automática de auditorías: Normativas como [PCI-DSS] (para manejo de tarjetas de crédito), [SOC 2] o [ISO 27001] prohíben estrictamente el almacenamiento de credenciales en texto plano dentro del código fuente. No usar una bóveda bloquea cualquier certificación internacional.	
Recomendación: Prohibir por completo las credenciales estáticas en el código e implementar una arquitectura de inyección de secretos en tiempo de ejecución (Runtime Secret Injection) utilizando una bóveda centralizada.	

Observación 13: Prueba de penetración o de regresión enfocados en la arquitectura de ciberseguridad	**Riesgo ALTO**
Condición: No posee / Inexistente	
Criterio: No posee / Inexistente	
Causa: No posee / Inexistente	
Efecto: 1. Riesgos de Exposición y Ataques Realistas • Vulnerabilidades de lógica y configuración: Herramientas como SAST o SCA (vistas anteriormente) solo leen código y dependencias. No pueden detectar si un atacante puede saltarse la autenticación manipulando los tokens en vivo, o si un puerto crítico de la base de datos quedó expuesto públicamente por un error de configuración en la red. • Falsas suposiciones de seguridad: El equipo de TI puede creer que el sistema está protegido por un firewall o un sistema de detección de intrusos (IDS), pero sin simulaciones de ataques reales, no sabrán si estas herramientas están correctamente configuradas o si están siendo ignoradas por completo. 2. Riesgos de Operación (El "Parchado Ciegas") • Reapertura de fallas antiguas (Regresión): Es común que una vulnerabilidad que ya fue corregida en el pasado vuelva a aparecer meses después porque un desarrollador utilizó una base de código desactualizada para crear una nueva función. Sin pruebas de regresión de seguridad, los errores del pasado regresan silenciosamente. • Falta de resiliencia ante incidentes: Si el departamento de TI no practica activamente cómo reaccionan sus sistemas ante un intento de intrusión, la primera vez que lo experimenten será durante un ataque real, lo que aumentará drásticamente el tiempo de respuesta y el daño colateral.	
Recomendación: Automatizar las pruebas de regresión de ciberseguridad en el pipeline (DAST) y programar auditorías periódicas de penetración (Pentesting) humanas, creando un enfoque de defensa en capas.	

INFORME PREVIO

Observación 14: Pruebas de Aceptación de Usuario (UAT)	**Riesgo ALTO**
Condición: No posee / Inexistente	
Criterio: No posee / Inexistente	
Causa: No posee / Inexistente	
Efecto: 1. El Síndrome del "En mi máquina sí funciona" (Falsos Positivos) • Configuraciones viciadas: Los entornos de desarrollo suelen tener librerías locales, permisos de administrador globales o bases de datos con datos de prueba modificados a mano. Una función puede pasar el UAT de forma exitosa en este entorno, pero fallar catastóficamente al llegar a producción porque carece de los permisos o configuraciones reales. • Interrupciones mutuas: Si un usuario está haciendo un UAT crítico y un desarrollador sube un cambio de código al mismo tiempo, el sistema se caerá o cambiará su comportamiento a mitad de la prueba, haciendo perder tiempo al negocio y generando frustración. 2. Riesgos de Seguridad y Confidencialidad de Datos • Contaminación de datos de negocio: Para que un UAT sea efectivo, los usuarios necesitan probar con flujos realistas. Si operan en el entorno de codificación, corren el riesgo de sobreescribir datos técnicos esenciales o, peor aún, los desarrolladores podrían tener acceso visual a información de negocio confidencial que el usuario capture durante sus pruebas. • Ausencia de pruebas de rendimiento reales: En el entorno de codificación no se puede medir la velocidad real del sistema ni cómo reacciona bajo estrés, ya que comparte recursos con herramientas de compilación y depuración de los programadores. 3. Riesgos de Gobierno y Cumplimiento (Compliance) • Violación de la Segregación de Funciones (SoD): Normativas internacionales (como ISO 27001, SOC 2 o auditorías financieras) exigen estrictamente que quienes escriben el código no sean los mismos que controlan el entorno donde se aprueba. No separar estos entornos bloquea cualquier certificación de calidad o seguridad para el departamento de TI.	
Recomendación: Establecer una arquitectura de entornos alineada con las mejores prácticas de la industria (Pipeline de 4 Entornos), garantizando q	

INFORME PREVIO

Observación 15: Anonimización o enmascaramiento de datos	**Riesgo ALTO**
Condición: No posee / Inexistente	
Criterio: No posee / Inexistente	
Causa: No posee / Inexistente	
Efecto: Riesgos de Cumplimiento Legal y Multas (La mayor amenaza) • Violación directa de leyes de privacidad: Regulaciones globales como el GDPR (Reglamento General de Protección de Datos) y normativas locales de protección de datos prohíben estrictamente el uso de Información Personal Identificable (PII) en entornos que no sean de producción sin una base legal explícita. • Sanciones financieras severas: El uso indebido de datos reales en pruebas es considerado una negligencia grave por los entes reguladores, lo que puede acarrear multas masivas que impactan directamente el flujo de caja de la empresa. 2. Riesgos de Ciberseguridad y Exposición de Datos • Entornos desprotegidos y vulnerables: Los entornos de pruebas típicamente carecen de bitácoras de auditoría estrictas, cifrado avanzado o monitoreo en tiempo real. Si un atacante vulnera la red de desarrollo, se lleva consigo el historial transaccional completo de los clientes reales. • Fuga de datos por el factor humano: El acceso masivo de personal técnico y proveedores externos a datos reales incrementa exponencialmente el riesgo de filtraciones internas (intencionadas o accidentales). 3. Riesgos Operativos y de Negocio (Errores en el mundo real) • Despliegue accidental de notificaciones reales: Si el entorno de pruebas tiene datos transaccionales verdaderos y un desarrollador ejecuta una prueba de correo electrónico o alertas SMS masivas por error, los clientes reales recibirán notificaciones de transacciones falsas, desatando pánico, colapso en el soporte técnico y un severo daño reputacional.	
Recomendación: Implementar una estrategia automatizada de Gestión de Datos de Prueba (Test Data Management - TDM) mediante herramientas que realicen un enmascaramiento estático e irreversible directamente en el pipeline de entrega de datos.	

Observación 16: Matriz formal de Segregación de Funciones (SoD - Segregation of Duties)	**Riesgo ALTO**
Condición: No posee / Inexistente	
Criterio: No posee / Inexistente	
Causa: No posee / Inexistente	
Efecto: 1. Fraude e Inyección de Código Malicioso (Riesgo Interno) • Modificaciones sin supervisión: Un programador con acceso de despliegue directo a producción podría alterar líneas de código para desviar centavos de transacciones hacia una cuenta externa, o deshabilitar intencionalmente un módulo de seguridad para vender la vulnerabilidad a terceros. • Ausencia de contrapesos: Si el que audita el sistema (QA/Seguridad) es el mismo que diseña el software, las fallas éticas o de código se ocultarán sistemáticamente para cumplir de forma artificial con los tiempos de entrega. 2. Errores Accidentales Fuera de Control • Despliegues en caliente e improvisados: Sin una separación de roles, los desarrolladores corregirán problemas directamente sobre el servidor de producción ("parches en vivo"). Un simple error de sintaxis o una consulta SQL mal optimizada puede corromper la base de datos central y paralizar la operación del negocio por días. 3. Incumplimiento e Invalidez ante Auditorías • Pérdida de certificaciones internacionales: La segregación de funciones es el pilar número uno exigido por marcos regulatorios y de calidad informática globales como ISO 27001, SOC 2, COBIT y normativas financieras como SOX (Sarbanes-Oxley) o [PCI-DSS]. Operar sin una matriz SoD formal formaliza la reprobación inmediata de cualquier auditoría corporativa.	
Recomendación: Desarrollar una matriz SOD	

INFORME PREVIO

4. Limitaciones al alcance (información no recibida)

No se registraron excepciones por información no recibida.

6. Conclusión

Resultado PRELIMINAR de la auditoría presencial practicada a SISTEMAS, la calificación obtenida es 68.4% (aceptable). Se identificaron 16 observaciones (15 de riesgo alto, 1 medio y 0 bajo), cuyas recomendaciones deben atenderse mediante el Plan de Acción Correctiva (PAC).

Anexo de evidencias

N°	Archivo	Descripción	Huella SHA-256
1	Cuestionario_Sistema_Empresa_TI.xlsx	Cuestionario	991acd9038bf8c4c…
2	MANUAL DE NORMAS Y PROCEDIMIENTOS GESTIO■N DE CAMBIOS_AGOSTO 2022 12082022.pdf	Manual	f7642d1638c93373…
3	Cuestionario_Sistema_Empresa_TI.xlsx	Cuestionario de Auditoría de Sistemas utilizado para cargar la estructura, respuestas y evidencias de la auditoría	991acd9038bf8c4c…

INFORME PREVIO